

Unit 4: Data Preprocessing

Data Transformation Techniques – Class Activity Sheet #7

Feature scaling, feature encoding, and feature engineering

Name		Batch	
Date		Global ID	

Learning Objectives

- Explain why data transformation is needed before model building.
- Apply feature scaling using Min-Max normalization and standardization.
- Convert categorical features into numerical features using encoding.
- Create new meaningful features using feature engineering.
- Implement each transformation technique using Python.

Part A: Concept Recall

Data transformation changes the format, scale, or representation of data so that it becomes more useful for analysis or machine learning.

Technique	Purpose	Example
Feature scaling	Bring numerical values to a comparable range	Scale salary and age
Feature encoding	Convert categories into numbers	Male/Female to 0/1
Feature engineering	Create new useful features	Total marks from subject marks

Part B: Feature Scaling

Feature scaling is used when numerical columns have different ranges. Many algorithms work better when features are on a similar scale.

Scaling Technique	Formula	Output Range / Meaning
Min-Max scaling	$x_{\text{scaled}} = (x - \min) / (\max - \min)$	Usually 0 to 1
Standardization	$z = (x - \text{mean}) / \text{standard deviation}$	Mean becomes 0, standard deviation becomes 1
Robust scaling	$x_{\text{scaled}} = (x - \text{median}) / \text{IQR}$	Useful when outliers are present

Activity B1: Manual Min-Max Scaling

Dataset: Marks = 50, 60, 70, 80

Minimum = _____ Maximum = _____

Formula: $x_{\text{scaled}} = (x - \min) / (\max - \min)$

Python code:

```
import pandas as pd
df = pd.DataFrame({"Age": [20, 30, 40, 50]})
df["Age_MinMax"] = (df["Age"] - df["Age"].min()) / (df["Age"].max() - df["Age"].min())
print(df)
```

Activity B2: Manual Standardization

Dataset: Marks = 50, 60, 70, 80

Mean = _____ Standard deviation = _____

Formula: $z = (x - \text{mean}) / \text{standard deviation}$

Marks	Min Max Scaling (x - min) / (max - min)	Z-score Standardization (x - mean) / standard deviation	Robust Scaling (x - median) / IQR
50			
60			
70			
80			

Python code using scikit-learn:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
df = pd.DataFrame({"Marks": [50, 60, 70, 80]})
scaler = StandardScaler()
df["Marks_Standardized"] = scaler.fit_transform(df[["Marks"]])
print(df)
```

Part C: Feature Encoding

Feature encoding converts categorical values into numerical values because most machine learning algorithms work with numbers.

Encoding Technique	Mathematical / Logical Idea	Best Used For
Label encoding	Assign each category an integer	Ordinal categories
One-hot encoding	Create binary columns with 0 or 1	Nominal categories
Ordinal encoding	Preserve meaningful order	Low, Medium, High
Binary encoding	Convert categories into binary representation	High-cardinality categories

Activity C1: Label Encoding

Convert the Gender column into numbers.

Student	Gender	Encoded Gender
A	Male	
B	Female	
C	Female	
D	Male	

Rule: Male = 1, Female = 0

Python code:

```
df = pd.DataFrame({"Gender": ["Male", "Female", "Female", "Male"]})
df["Gender_Encoded"] = df["Gender"].map({"Female": 0, "Male": 1})
print(df)
```

Activity C2: One-Hot Encoding

Convert the City column into separate binary columns.

Student	City	City_Delhi	City_Mumbai	City_Chennai
A	Delhi			
B	Mumbai			
C	Chennai			
D	Delhi			

Python code:

```
df = pd.DataFrame({"City": ["Delhi", "Mumbai", "Chennai", "Delhi"]})
encoded_df = pd.get_dummies(df, columns=["City"], dtype=int)
print(encoded_df)
```

Activity C3: Ordinal Encoding

Convert Education Level into ordered numbers.

Person	Education Level	Encoded Value
P1	School	
P2	Graduate	
P3	Postgraduate	
P4	Graduate	

Rule: School = 1, Graduate = 2, Postgraduate = 3

Python code:

```
df = pd.DataFrame({"Education": ["School", "Graduate", "Postgraduate", "Graduate"]})
order = {"School": 1, "Graduate": 2, "Postgraduate": 3}
df["Education_Encoded"] = df["Education"].map(order)
print(df)
```

Part D: Feature Engineering

Feature engineering creates new columns from existing columns. A well-engineered feature can make patterns easier for a model to learn.

Technique	Formula / Logic	Example
Addition feature	Total = $x_1 + x_2 + x_3$	Total marks
Ratio feature	Ratio = x_1 / x_2	Price per square foot
Difference feature	Difference = $x_1 - x_2$	Profit = Sales - Cost
Date feature	Extract day, month, year	Month from purchase date
Binning	Convert continuous values into groups	Age group

Activity D1: Create Total and Average Marks

Student	Maths	Science	English	Total	Average
A	70	80	75		
B	60	65	70		
C	90	85	95		

Formulas: Total = Maths + Science + English; Average = Total / 3

Python code:

```
df = pd.DataFrame({
    "Maths": [70, 60, 90],
    "Science": [80, 65, 85],
    "English": [75, 70, 95]
})
df["Total"] = df["Maths"] + df["Science"] + df["English"]
df["Average"] = df["Total"] / 3
print(df)
```

Activity D2: Extract Date Features

Create Year, Month, and Day from a Date column.

Python code:

```
df = pd.DataFrame({"Purchase_Date": ["2026-01-15", "2026-02-20", "2026-03-05"]})
df["Purchase_Date"] = pd.to_datetime(df["Purchase_Date"])
```

```
df["Year"] = df["Purchase_Date"].dt.year
df["Month"] = df["Purchase_Date"].dt.month
df["Day"] = df["Purchase_Date"].dt.day
print(df)
```

Part E: Combined Practice Dataset

Apply scaling, encoding, and feature engineering on the following dataset.

Student	Age	Gender	City	Maths	Science	English
A	20	Female	Delhi	70	80	75
B	22	Male	Mumbai	60	65	70
C	21	Female	Chennai	90	85	95
D	23	Male	Delhi	55	60	58

1. Scale the Age column using Min-Max scaling.
2. Encode the Gender column using label encoding.
3. Encode the City column using one-hot encoding.
4. Create Total marks and Average marks.
5. Which new feature may be useful for predicting student performance?

Python code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
df = pd.DataFrame({
    "Age": [20, 22, 21, 23], "Gender": ["Female", "Male", "Female", "Male"],
    "City": ["Delhi", "Mumbai", "Chennai", "Delhi"], "Maths": [70, 60, 90, 55],
    "Science": [80, 65, 85, 60], "English": [75, 70, 95, 58]
})
scaler = MinMaxScaler()
df["Age_scaled"] = scaler.fit_transform(df[["Age"]])
df["Gender_encoded"] = df["Gender"].map({"Female": 0, "Male": 1})
df = pd.get_dummies(df, columns=["City"], dtype=int)
df["Total"] = df["Maths"] + df["Science"] + df["English"]
df["Average"] = df["Total"] / 3
print(df)
```

After the code is executed. Guess the output:

Part F: Combined Practice Dataset-2

Employee	Age	Gender	Department	Experience	Training Hours	Projects Completed
E1	24	Female	HR	2	15	4
E2	30	Male	IT	6	25	8
E3	27	Female	Finance	4	20	6
E4	35	Male	IT	10	30	12
E5	29	Female	HR	5	18	7
E6	32	Male	Finance	8	28	10

Tasks(Write Code)

1. Create the given dataset as a Pandas DataFrame.
2. Display the first five rows, dataset shape, column names, and data types.
3. Display the statistics on the data.
4. Scale the Age column using Min Max scaling and store the result in Age Scaled.
5. Standardize the Experience column using StandardScaler and store the result in Experience Standardized.
6. Encode the Gender column using label encoding and store the result in Gender Encoded.
7. Encode the Department column using one hot encoding.
8. Create Productivity Score using: Training Hours + (2 x Projects Completed).
9. Create Experience Level: 0 to 3 years as Beginner, 4 to 7 years as Intermediate, and 8 years or more as Experienced.
10. Display the completely transformed dataset.

Analysis Questions

1. What is the scaled age of employee E4?
2. What numerical values were assigned to Female and Male during label encoding?
3. How many columns were created through one hot encoding of Department? Name them.
4. Which employee has the highest Productivity Score? Show the calculation.
5. Which features could be useful for predicting employee performance? Explain your choice.
6. Why should an employee identifier generally not be used as a predictive feature?

Conclusion: Data transformation prepares raw data into a numerical and meaningful form suitable for machine learning models.